

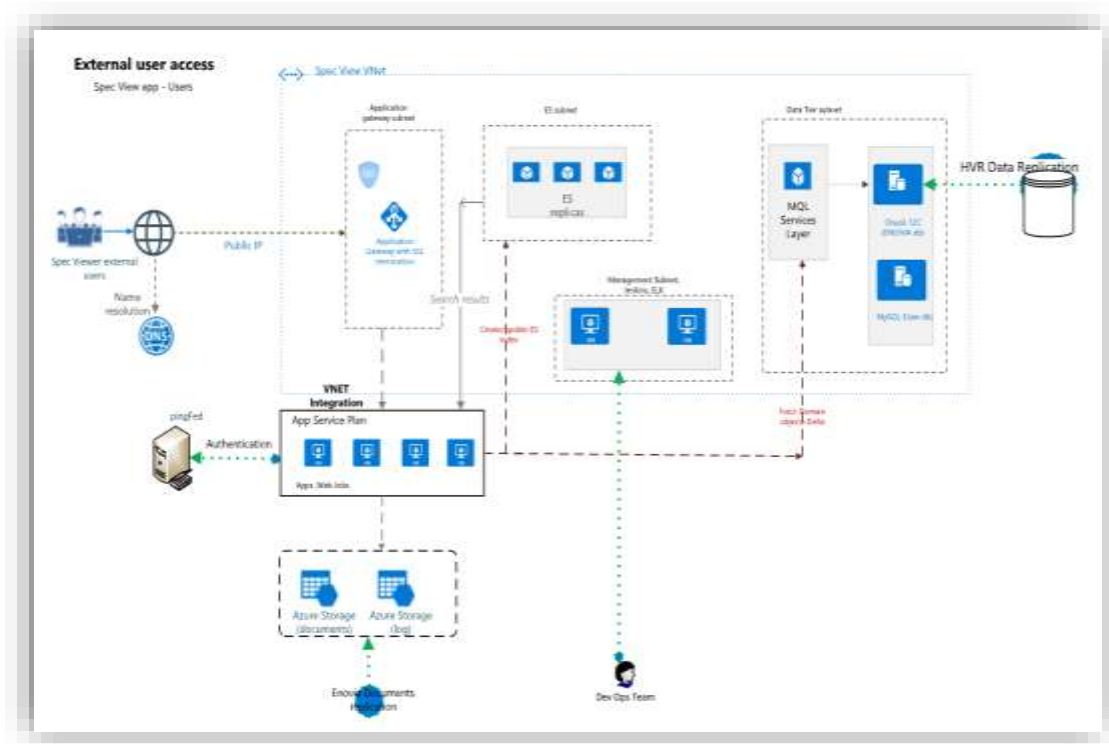
TECHNICAL SPECIFICATION

Issued Date :30/01/2020		SpecsAnywhere Technical Specification
Version	Date	Description of changes
1.0	30/01/2020	Initial Version
2.0	05/03/2020	Update Initial Load, Delta, Elastic and ETL Automation
3.0	11/03/2020	User Profile Extraction and architecture for security with security matrix
4.0	18/03/2020	FCS update, modified Initial load & Delta design Security (jwtTokens)
5.0	04/07/2020	Rclone – mount and FSCrawler updates for document pipeline
6.0	09/07/2020	Data Pipeline setup
Responsible		
Developers:		Reviewer:
Infosys Team		Infosys

1. Architecture

The Architecture for SpecAnywhere is presented below.

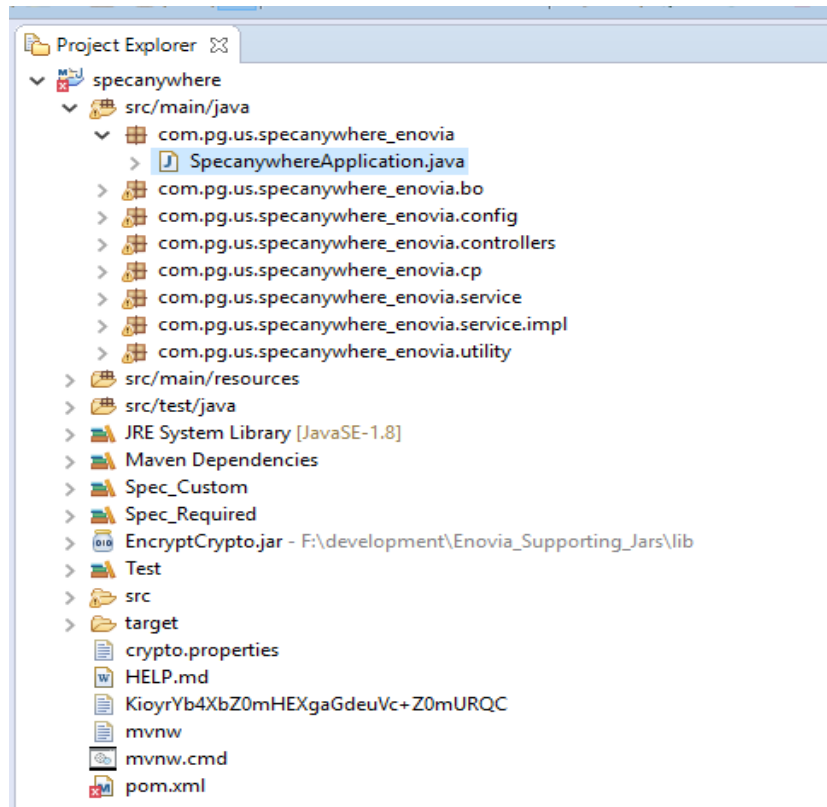
The overall high-level goal at this time is to be able to present *Read-only* data to the end user from a HVR copy of the Enovia database using state-of-the-art technology.



2. Design Overview:

- This project must be developed using spring boot framework micro services technology.
- Elastic-search is the choice of Search Engine and is an open-source technology used for this project.
- Following is the complete package From Enovia

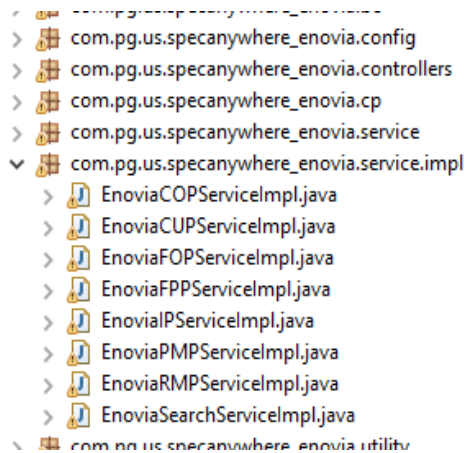
TECHNICAL SPECIFICATION



3. Technical Design Break Down

- The project is designed based on spring-boot framework and as a result needs to follow the above project structure.
- The purpose of each package is mentioned below:
 - The root class *Specanywhere_Application.java* is the service class that boots the project.
 - Controller Package holds the controller class that invokes each service implementation based on enovia type for data extraction.
 - Service.impl* is the implementation package for extracting data from enovia and expose data to microservices. The context must always be maintained as sys_ops as the administrator user. The credentials must be encrypted as per the P&G standards.

TECHNICAL SPECIFICATION



- Utility Package can contain every supporting class that helps the Business Layer

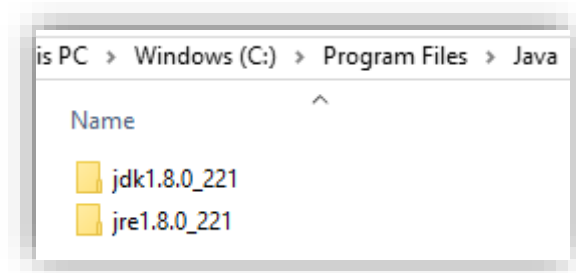
4. Installation

The following are software necessary for building this custom application:

- Java jdk1.8.0_221
- Oracle Server (Db Schema)
- enoviaV6R2018x (LivecollaborationServer + HF12)
- Enovia Embedded Tomcat
- Elastic (elasticsearch7.5.0)
- Postman (v7.19.1)

JAVA

Follow the standard jdk installation process. Install the Enovia supported JDK compatible version JDK 1.8.0_221 as shown below.



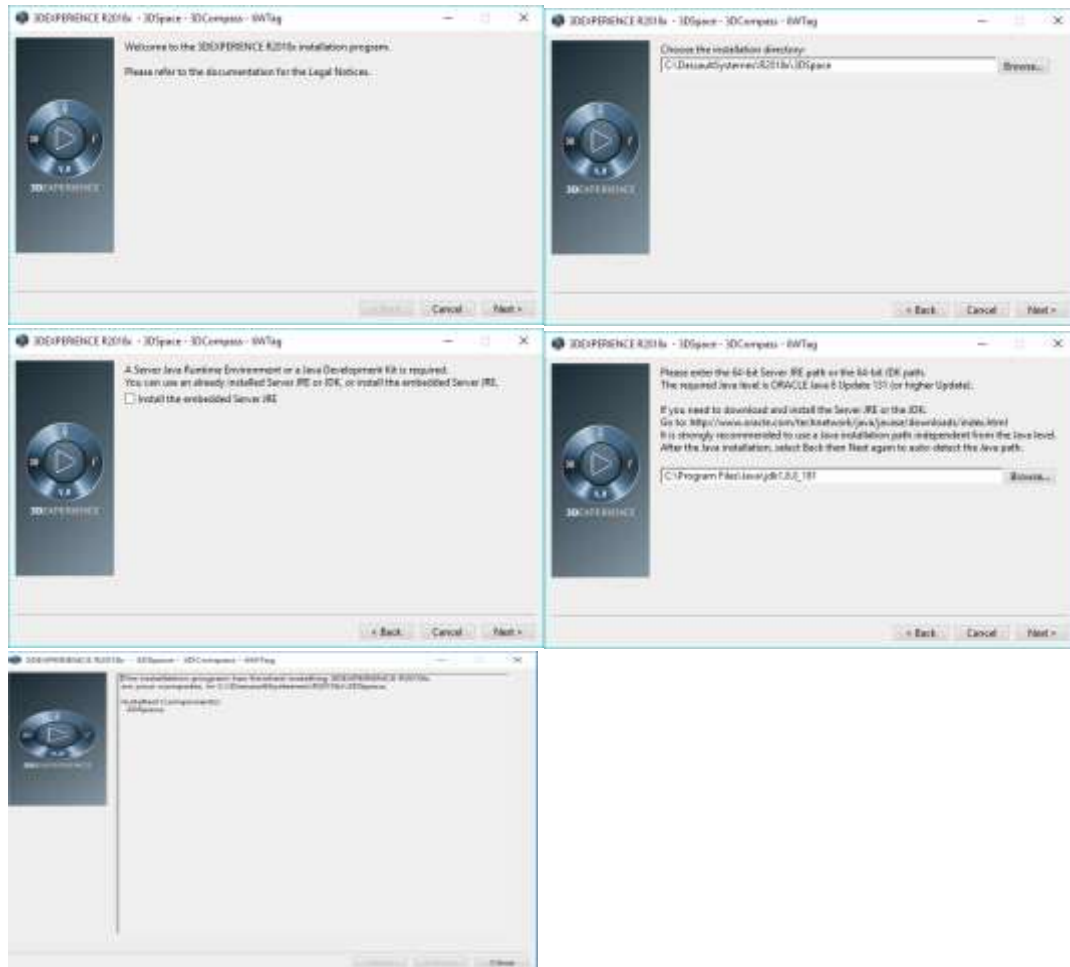
Oracle Server

This will be taken care by the DBA team

Enovia (3D Experience)

Provide all necessary information for the standard installation of Live-collaboration Server and 3D Space. Some sample snapshot here for reference.

Note: these snapshots DO NOT cover the complete installation process.



While *LivecollaborationServer* installable is sufficient for access to the DB layer, it is important to install 3DSpace to obtain the “*framework.jar*” that becomes the base for implementing the Business and DAL for this application.

Elastic

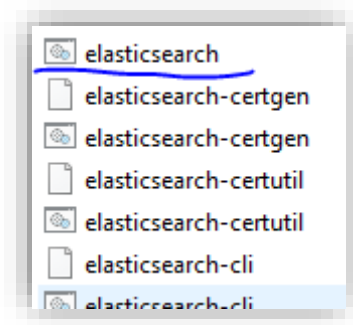
Download the elastic version from online if using an open-source version. For premium usage please refer Microsoft Azure portal.

elasticsearch-7.5.0

To start Elasticsearch service simply run the Elasticsearch.bat file if on windows:

F:\Elastic\elasticsearch-7.5.0\bin

The unzipped elastic project would look like this:



Postman

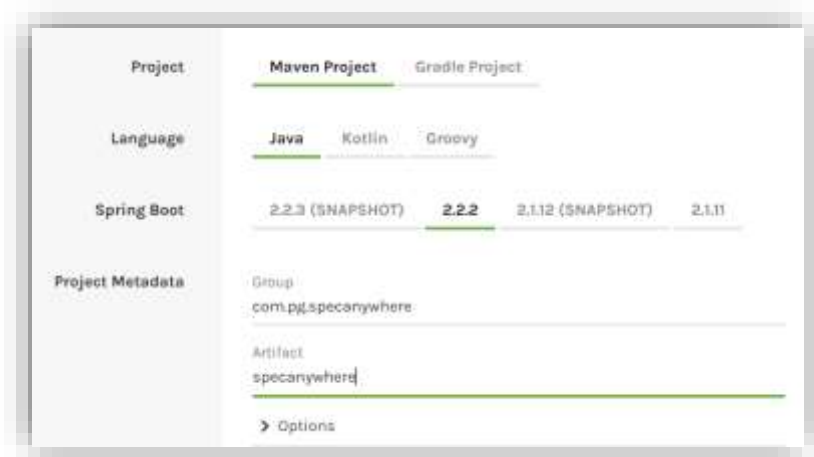
Postman can be installed to create schema, test elastic service as an alternative to Kibana and also verify data during initial load or delta respectively.

Project set up guidelines:

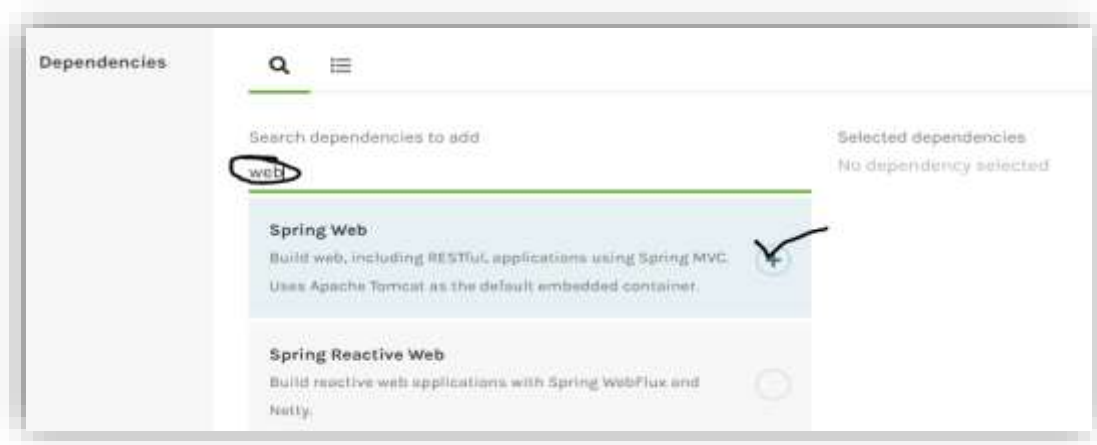
Go to this link:

<https://start.spring.io/>

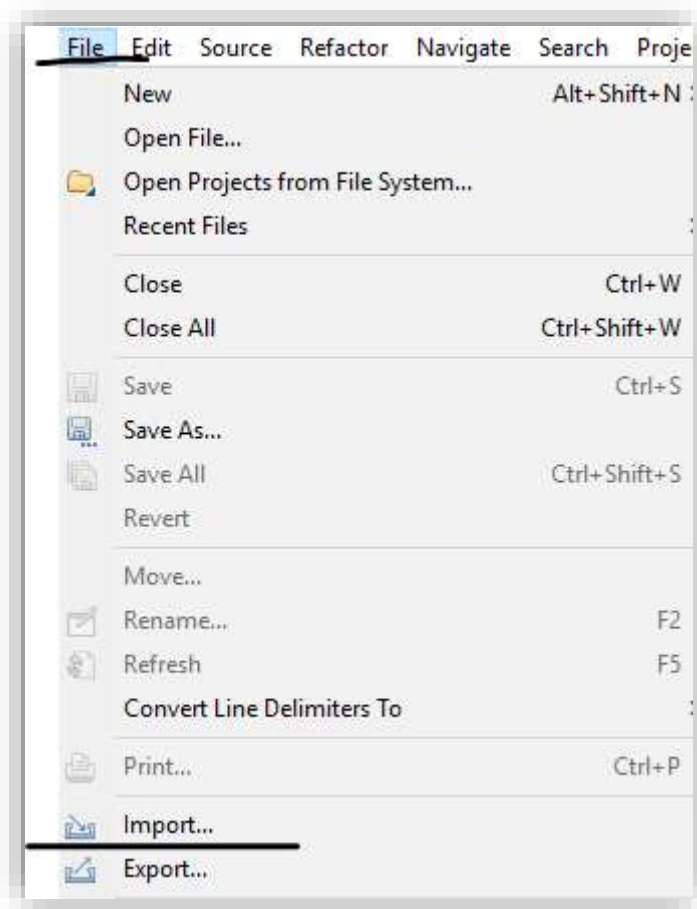
Choose options as mentioned below:



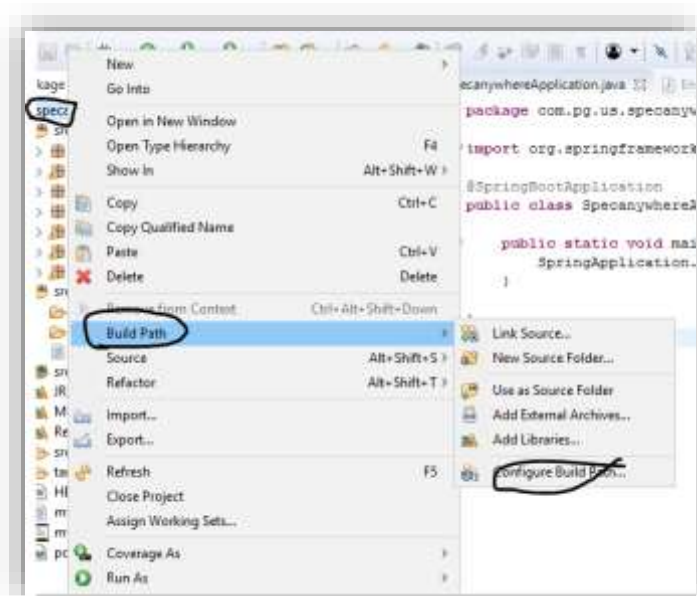
TECHNICAL SPECIFICATION

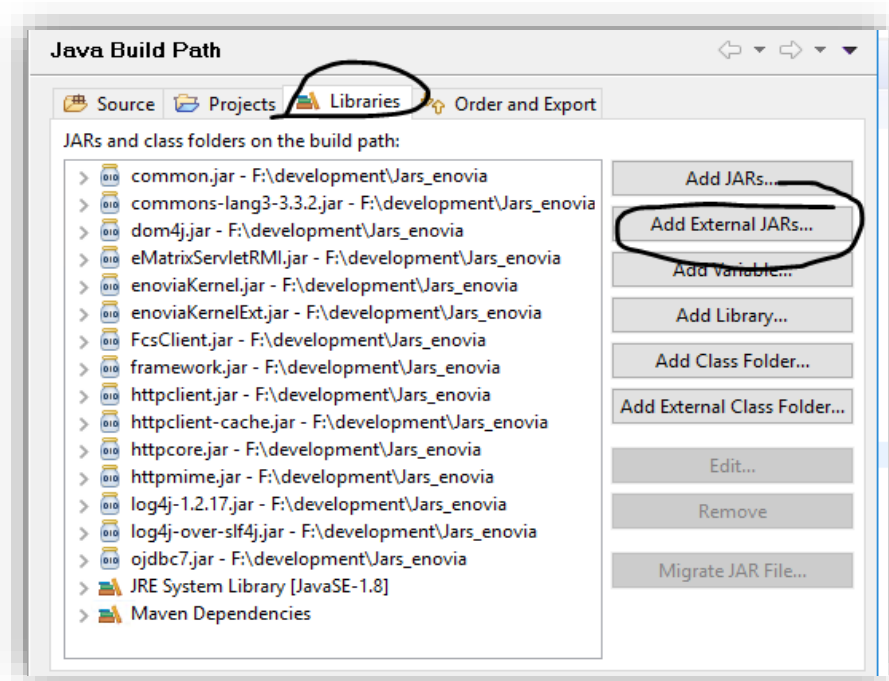


TECHNICAL SPECIFICATION



Import the maven project that was downloaded in the earlier snapshot.





- As an alternative, setup pom.xml with all the necessary jars. **(Recommended)**
As shown below:

```
<?xml version="1.0" encoding="UTF-8"?>
<project xmlns="http://maven.apache.org/POM/4.0.0" xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://maven.apache.org/xsd/maven-4.0.0.xsd">
<modelVersion>4.0.0</modelVersion>
<parent>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-parent</artifactId>
<version>2.2.2.RELEASE</version>
<relativePath/> <!-- lookup parent from repository -->
</parent>
<groupId>com.pg.us.specanywhere_enovia</groupId>
<artifactId>specanywhere</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>specanywhere</name>
<description>specanywhere</description>

<properties>
<java.version>1.8</java.version>
</properties>

<dependencies>
<dependency>
<groupId>org.springframework.boot</groupId>
<artifactId>spring-boot-starter-web</artifactId>
</dependency>

<dependency>
<groupId>com.pg.common</groupId>
<artifactId>enovia-common</artifactId>
<version>1.0</version>
</dependency>

<dependency>
<groupId>com.pg.commonlang3</groupId>
<artifactId>enovia-common-lang3</artifactId>
<version>3.3.2</version>
</dependency>
</dependencies>
```

```

<dependency>
  <groupId>com.pg.cpnaccelerator</groupId>
  <artifactId>enovia-cpnaccelerator</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.dom4j</groupId>
  <artifactId>enovia-dom4j</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.eMatrixServletRMI</groupId>
  <artifactId>enovia-eMatrixServletRMI</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.enoviaKernel</groupId>
  <artifactId>enovia-enoviaKernel</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.enoviaKernelExt</groupId>
  <artifactId>enovia-enoviaKernelExt</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.enterprisechangemgt</groupId>
  <artifactId>enovia-enterprisechangemgt</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.FcsClient</groupId>
  <artifactId>enovia-FcsClient</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.formulation</groupId>
  <artifactId>enovia-formulation</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.framework</groupId>
  <artifactId>enovia-framework</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.httpclient</groupId>
  <artifactId>enovia-httpclient</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.httpcore</groupId>
  <artifactId>enovia-httpcore</artifactId>
  <version>1.0</version>
</dependency>

<dependency>
  <groupId>com.pg.httpmime</groupId>
  <artifactId>enovia-httpmime</artifactId>

```

```

        <version>1.0</version>
    </dependency>

    <dependency>
        <groupId>com.pg.servlet-api</groupId>
        <artifactId>enovia-servlet-api</artifactId>
        <version>1.0</version>
    </dependency>

    <dependency>
        <groupId>com.pg.log4j</groupId>
        <artifactId>enovia-log4j</artifactId>
        <version>1.2.17</version>
    </dependency>

    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
        <exclusions>
            <exclusion>
                <groupId>org.junit.vintage</groupId>
                <artifactId>junit-vintage-engine</artifactId>
            </exclusion>
        </exclusions>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

Clean project in eclipse and the jars will be downloaded automatically from the maven repository.

Please note you need to manually add the Enovia jars to your local maven repository for the first time to make this work.

DB Connectivity:

The Enovia DB connectivity can be done using the Enovia API with the administrator credentials.

```

try {
    context = new Context("");
    context.setUser(USER);
    context.setPassword(PASS);
    context.setVault(VAULT);
    context.connect();
}

```

The credentials encryption can be done using P&G's standard library:

```
PASS = EncryptCrypto.decryptString(PASS);
```

TECHNICAL SPECIFICATION

Controller Service Configurations:

The annotation RestController/Mapping needs to be defined to specify the enovia service.

```
@RestController(value = "/enovia-api")
@RequestMapping(value = "/enovia-api")
@CrossOrigin(origins = "*")
```

Auto wire the enovia context with the following annotation:

```
@Autowired
Environment env;
```

For each Implementation service something like the following should be written in the controller class:

```
@GetMapping("/search/{searchId}")
public ResponseEntity<String> getSearchResult(@PathVariable(value = "searchId") String searchString)
    throws Exception {
    List<SearchBO> list = search.getSearchResult(searchString, env);
    ObjectMapper mapperObj = new ObjectMapper();
    String json = null;
    try {
        json = mapperObj.writeValueAsString(list);
    } catch (JsonProcessingException e) {
        e.printStackTrace();
        throw e;
    }
    return ResponseEntity.ok(json);
}
```

Business Object Package:

Create separate BO - Business Object classes for each type and set all constants required to be extracted from the Enovia base object.

All business object level attributes needs to be consolidated into one selectable.

Example:

```
public static Map<String, StringList> getFppRelatedObjsSelectableMap(Context co)
{
    StringList busSelect = new StringList();
    StringList relSelect = new StringList();
    Map<String, StringList> mapSelectables = new HashMap<String, StringList>();
    busSelect.addAll(BusObjectAttribUtil.getBasicInfoBusSelectable(context));
    busSelect.addAll(getCommonDataBusSelectable(context));
}
```

This would mean that a getInfo on this selectable would fetch all data in a single database hit.

For the more complex relational structure there needs to be carefully designed *getRelated* enovia API methods based on level of relational data extraction.

Example:

TECHNICAL SPECIFICATION

```
mlRelatedObjects = doFPP.getRelatedObjects(context,  
    ConstantsUtil.RELATIONSHIP_CHARACTERISTIC,  
    ConstantsUtil.TYPE_PGTRANSPORTUNITCHARACTERISTIC+", "+(
```

The final result-set must be a *hashmap* that will be converted to JSON dynamically in the controller class.

Setup the local server port number in the application.properties file:



To start service simple execute the SpecAnywhere root application java file generated by spring boot as shown below:

```
Spring Boot (v2.2.2.RELEASE)

2020-01-31 14:07:25.309 INFO 15472 --- [main] c.p.u.s.SpecanywhereApplication : Starting Spe
2020-01-31 14:07:25.214 INFO 15472 --- [main] c.p.u.s.SpecanywhereApplication : No active pr
2020-01-31 14:07:26.596 INFO 15472 --- [main] o.s.b.w.embedded.tomcat.TomcatWebServer : Tomcat initi
2020-01-31 14:07:26.350 INFO 15472 --- [main] o.apache.catalina.core.StandardService : Starting ser
2020-01-31 14:07:26.550 INFO 15472 --- [main] org.apache.catalina.core.StandardEngine : Starting Ser
2020-01-31 14:07:26.665 INFO 15472 --- [main] o.s.c.o.C.[Tomcat].[localhost].[/] : Initializing
2020-01-31 14:07:26.668 INFO 15472 --- [main] o.s.web.context.ContextLoader : Root WebAppl

*INITIALIZING CONTEXT*
```

This will host all enovia TYPE services on the following URL for example:

<http://localhost:9091/enovia-api/fpp/20336.41905.36548.59902>

To test your service simple execute the above URL in a browser and the result should be a JSON with all the extracted data.

Initial and Delta load:

Count of TYPES (within POC scope) that need to be extracted for each Delta averages around these numbers below:

TECHNICAL SPECIFICATION

RELEASED OBJECTS

MQL<16>eval expr 'count true' on temp query bus "Finished Product Part" * * wh\$
12713

MQL<17>eval expr 'count true' on temp query bus "pgConsumerUnitPart" * * where\$
1727

MQL<19>eval expr 'count true' on temp query bus "pgCustomerUnitPart" * * where\$
1860

MQL<20>eval expr 'count true' on temp query bus "Raw Material" * * where 'modi\$
2334

MQL<21>eval expr 'count true' on temp query bus "Packaging Material Part" * * \$
5841

All types of objects in DB that might need to be ETL into elastic or other custom DB:

ALL OBJECTS + RELEASE (24hr)

MQL<29>eval expr 'count true' on temp query bus * * * where 'modified>02/11/20\$
59,982

ALL OBJECTS + RELEASE (48hr)

MQL<25>eval expr 'count true' on temp query bus * * * where 'modified>02/10/20\$
81,639

ALL OBJECTS + RELEASE (24hr) – ON RANDOM DAYS

MQL<35>eval expr 'count true' on temp query bus * * * where 'modified>02/12/20\$
7,453

MQL<36>eval expr 'count true' on temp query bus * * * where 'modified<02/12/20\$
53,541

MQL<37>eval expr 'count true' on temp query bus * * * where 'modified<02/11/20\$
21,790

MQL<38>eval expr 'count true' on temp query bus * * * where 'modified<02/11/20\$
0

MQL<39>eval expr 'count true' on temp query bus * * * where 'modified<02/11/20\$
21,790

MQL<40>eval expr 'count true' on temp query bus * * * where 'modified<02/10/20\$
1253

MQL<41>eval expr 'count true' on temp query bus * * * where 'modified<02/09/20\$
3971

TECHNICAL SPECIFICATION

MQL<42>eval expr 'count true' on temp query bus * * * where 'modified<02/08/20\$
13,164

Several timings have been captured on various months over a 1 year period and the highest number of modified objects have been captured in a 24 hour period.



Delta load must be extracted such that only “Release” and “Obsolete” data be considered. This will set the old revisions to obsolete.

The delta load and the initial load must comprise of all attributes necessary for search, file-download and security classification.

Based on Enovia security analysis based on the security framework shared by P&G and OOTB understanding of Enovia 3D Experience the flow for security matrix has been drafted below.



Security_Flow_V1.xl
sx

While the above is the general rule for specsanywhere, there are other requirements which supersede this security filter.

The initial Load script must be separate for each type (only types in scope of the project).

A sample script can be found here, which shows all the attributes that are required in the export.



The extraction must dump this data into a flat-file on specified folder.

The tcl script must be triggered from a batch file which in turn must be referenced in a master batch script.

Here are 2 samples:



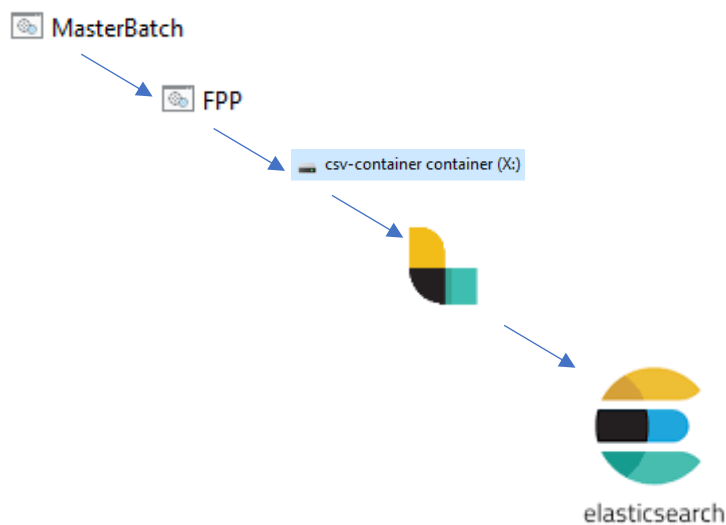
MasterBatch.bat



FPP.bat

The generated extract/flat-file must be moved to the STAGING folder for the listener to poll the event.

Here is the work-flow for the Initial/Delta Load:



The actual architecture implementing the data pipeline from Enovia to Specs-Anywhere is given descriptively below:

The batch scripts mentioned above will be scheduled on VM to generate the flat-files into mapped-drive or azure storage. And the Logstash server will poll the mounted directory for updated data into Elasticsearch index.



The initial load today can go up to a duration of 24 hours on development environment with only 1 instance of Logstash and Elasticsearch.

Elastic Setup and Implementation

Installation:

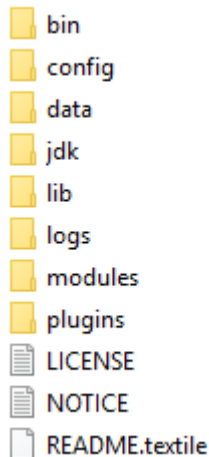
Elasticsearch can be installed by downloading the latest version of elastic from the Elasticsearch website namely:

<http://elastic.co/downloads/Elasticsearch>

A zipped version gets downloaded and the extract can be unzipped and placed in a dedicated Elastic folder, example:

F:\Elastic\elasticsearch7.5.0\elasticsearch-7.5.0

The product components will look similar to the snapshot shown below:



Elastic components must be included in the POM.xml to facilitate usage of elastic API. Here are the dependencies for elastic necessary for this initial load:

```
<dependency>
  <groupId>org.elasticsearch</groupId>
  <artifactId>elasticsearch</artifactId>
</dependency>
<dependency>
  <groupId>org.elasticsearch.client</groupId>
  <artifactId>elasticsearch-rest-high-level-client</artifactId>
  <version>7.5.1</version>
</dependency>
```

Schema:

The Schema used to build the Elastic Index is shown below:

PUT /apspec

```
{
  "settings": {
    "index": {
      "number_of_shards": 1,
      "number_of_replicas": 2
    },
    "analysis": {
      "filter": {
        "autocomplete_filter": {
          "type": "edge_ngram",
          "min_gram": 1,
          "max_gram": 20
        }
      },
      "analyzer": {
        "autocomplete": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": [
            "lowercase",
            "autocomplete_filter"
          ]
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "id": {
        "type": "keyword"
      },
      "type": {
        "type": "text",
        "analyzer": "whitespace"
      },
      "name": {
        "type": "text",
        "analyzer": "standard",
        "search_analyzer": "autocomplete"
      },
      "revision": {
        "type": "keyword"
      },
      "title": {
        "type": "text",
        "analyzer": "standard",
        "search_analyzer": "autocomplete"
      },
      "description": {
        "type": "text",
        "analyzer": "standard"
      },
      "originated": {
        "type": "date",
        "format": "M/d/yyyy h:m:ss a"
      },
      "modified": {
        "type": "date",
        "format": "M/d/yyyy h:m:ss a"
      },
      "ipClass": {
        "type": "keyword"
      },
      "classifiedItem": {
```

TECHNICAL SPECIFICATION



```

        "type": "text"
    },
    "company": {
        "type": "text"
    },
    "plant": {
        "type": "text"
    },
    "ownership": {
        "type": "text"
    },
    "filename": {
        "type": "text"
    },
    "filepath": {
        "type": "text"
    },
    "relatedspec": {
        "type": "text"
    },
    "masterspec": {
        "type": "text"
    },
    "phase": {
        "type": "text"
    },
    "releasedate": {
        "type": "date",
        "format": "M/d/yyyy h:m:ss a"
    },
    "brand": {
        "type": "text"
    },
    "displaytype": {
        "type": "text"
    },
    "tag": {
        "type": "text"
    },
    "originator": {
        "type": "keyword"
    },
    "orgsource": {
        "type": "text"
    },
    "stns": {
        "type": "text"
    },
    "ShareWithSupplier": {
        "type": "boolean"
    },
    "gendoc": {
        "type": "text"
    },
    "nonrenderabledoc": {
        "type": "text"
    },
    "otherrefdoc": {
        "type": "text"
    }
}
}
}

```

Elastic will be configured with 3 shards and 2 replicas respectively.

Business Logic

The main business logic associated to the Initial load will be implemented in the watcher service under a separate method call to CSVBulkInsert respectively.

- For the bulk import it is advisable to use RestHighLevelClient.

Example:

- RestHighLevelClient client = new RestHighLevelClient(RestClient.builder(new HttpHost(HOST, PORT, HTTP)));

- Read the input file that was exported using the tcl script.
- Use XContentBuilder xContentBuilder = jsonBuilder() for storing the extract as variables in the XContentBuilder.
- Use IndexRequest to create an index for the data to be pushed into ES. Example:
IndexRequest indexRequest = new IndexRequest(INDEX);
- Commit the bulk index requests such that only 100 or less are committed at a given time since multiple services will be running in parallel and to avoid any timeout requests.

```
if(count==100){
    client.bulk(bulkRequest, RequestOptions.DEFAULT);
    bulkRequest = new BulkRequest();
    count=1;
}
```

NOTE: It was initially at around 1000 bulk requests, but this can result in memory issues if there are parallel threads each running at 1000 capacity. It was reduced to 100 based on the current 30 parallel thread extraction logic. This may be reduced further if required (but this can have its own performance issues on initial load).

The business logic has been modified to make data flow more real time for delta. And Logstash has been configured to enable the flow from Azure blob to Logstash and then to Elasticsearch. The configuration and setup will be explained in greater detail in a later section of this document.

Listener Service

The listener service is a standalone java watcher service that comes with Java 8 and is capable of listening to event changes in a directory, sub-directory or folder and capture the same.

This listener service will perform 2 actions:

TECHNICAL SPECIFICATION

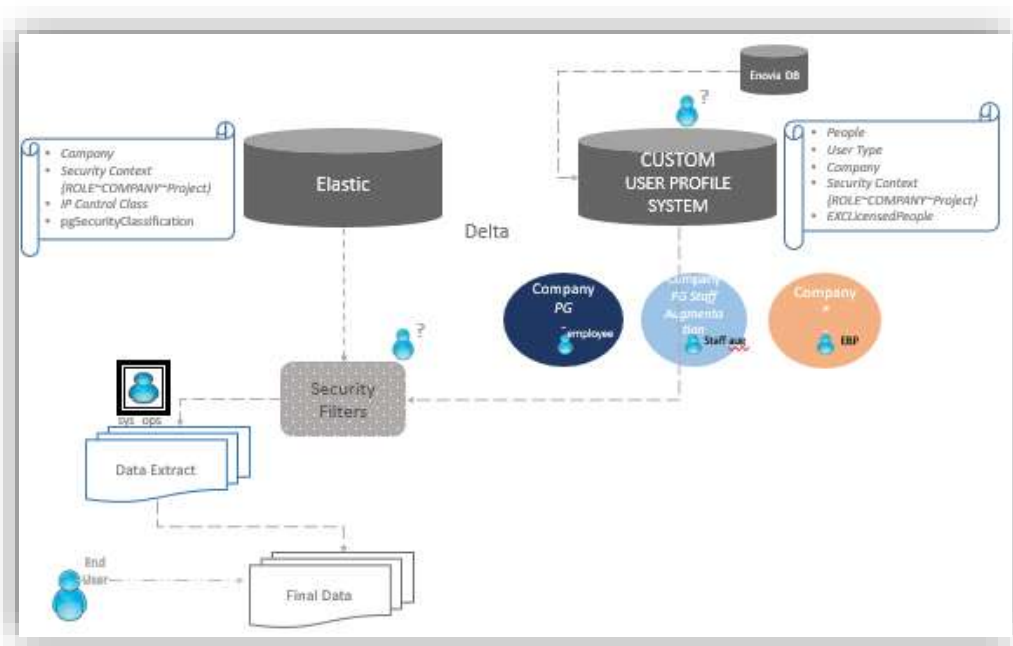
- Look for CREATE events for the configured directory
- Fire the Elastic Bulk Insert Service in a separate thread

Watcher Service Example:

Once the Batch Processor mentioned in the Initial Load/Delta section automatically places a copy of the data extract into the STAGING folder, the listener picks up the file and sends it for publishing to ES.

User Profile Extraction

The user profile information from Enovia needs to be moved out of it to validate the user profile against the Object security classification available in Elastic. In order to do so, a batch scheduled job is implemented which is essentially a TCL script that extracts User profile information and its security classification into Flat files which are then dumped into an SQL database.



The above is the sample architecture that explains how the security classification of the object is compared with that of the user security to arrive at access for a user for an object searched at elastic search.

TECHNICAL SPECIFICATION

FCS File download service

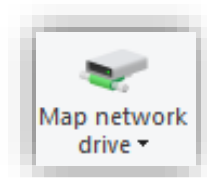
The FCS file download service is designed such that there are 2 pre-requisites:

- FCS HVR copy from production (an exact replica such that there is no change w.r.t the server name and folder structure (including naming and access control)).
- Azure Storage Account + Premium File Sharing enabled

Map drive of the FCS location to Azure File Share such that the copy is available in Azure.

How to map a drive to Azure file storage on Windows?

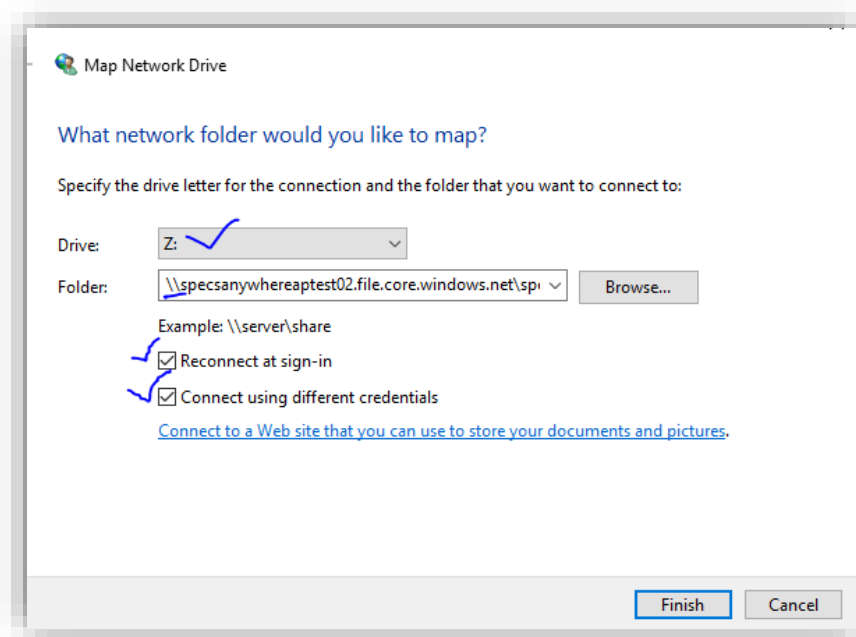
STEP1



STEP2

Provide the File share URI:

\\specsanywhereaptest02.file.core.windows.net\specsanywhere-ap-fcs



TECHNICAL SPECIFICATION

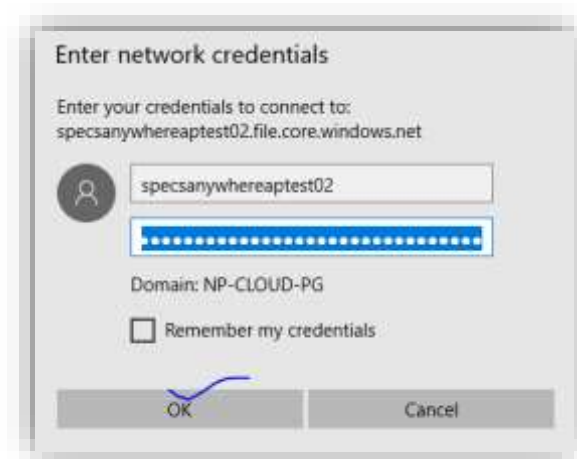
STEP3

Enter the Storage account name and key as shown here for example:

USERID: specsanywhereaptest02

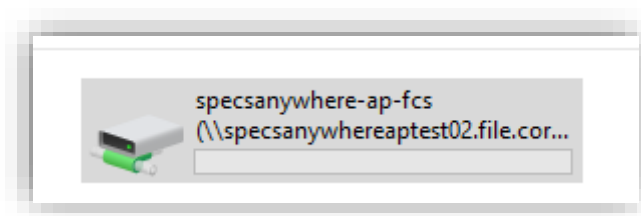
PWD:

dSqqueqweWdKyxCszVVibccn3dakdbn1uZWobmJVT+MojwTeHLRRYO+G2xkkGnt45X
AUhkw0Fq9nOBWUTPg=



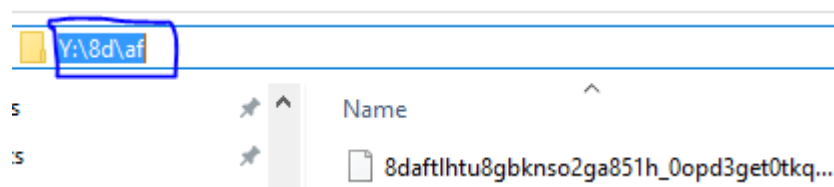
STEP4

New drive created



STEP5

The folder path can be seen as defined during creation and highlighted below.



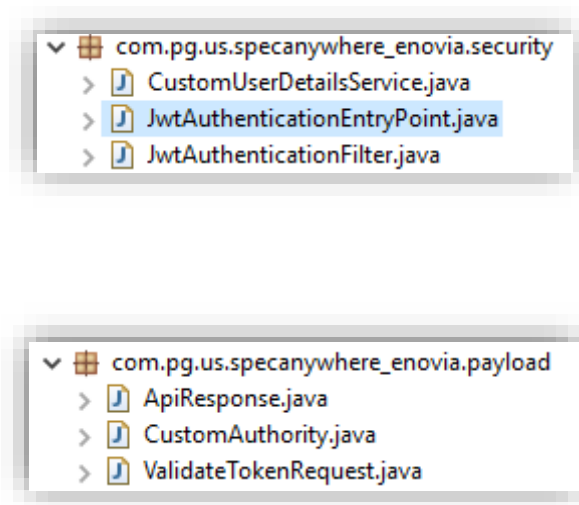
NOTE:

Due to P&G security rules the above logic has been decommissioned and the use of open-source technology RClone has been chosen to implement this pipeline.

Security Token for Enovia Service

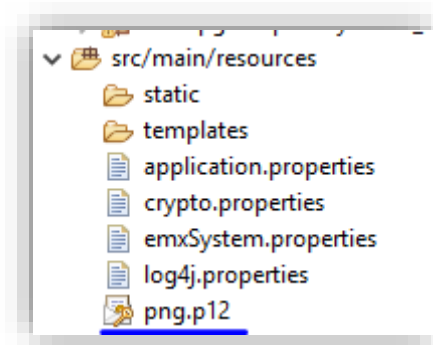
The security implementation is to secure the Enovia services layer with jwtTokens.

The project structure would be as such:



While most of these are mostly getters and setters the actual validation or filter based on jwtToken happens in JWTAuthenticationFilter.java

Some of the resources required for this are mentioned below:



application.properties shall be updated with these keys respectively:

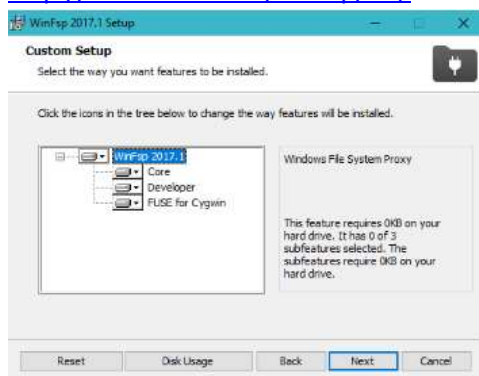
TECHNICAL SPECIFICATION

```
server.ssl.enabled=true
server.ssl.key-store-type=PKCS12
server.ssl.key-store=classpath:png.p12
server.ssl.key-store-password=sslsecret
server.ssl.trust-store-type=PKCS12
server.ssl.client-auth=want

zuul.routes.search-api.custom-sensitive-headers=true
zuul.sensitiveHeaders:Cookie,Set-Cookie
```

RClone:

1. download rclone for the respective OS (here we cover windows as an example)
<https://rclone.org/downloads/>
2. Extract to local directory (say: F:\rclone\)
3. Configure classpath to rclone.exe
4. Open cmd, type: rclone -version to test installation
5. Install WinFsp which is required to mount drive on windows for Blob with Rclone
6. <http://www.secfs.net/winfsp/rel/>



7. Open cmd from normal user prompt and not administrator and type:

F:\rclone\rclone-v1.52.2>**rclone config**

2020/07/04 09:14:19 NOTICE: Config file

"C:\\Users\\boindala.y\\.config\\rclone\\rclone.conf" not found - using defaults

No remotes found - make a new one

n) New remote

s) Set configuration password

q) Quit config

n/s/q> **n**

TECHNICAL SPECIFICATION

name> sample_container

Type of storage to configure.

Enter a string value. Press Enter for the default ("").

Choose a number from below, or type in your own value

1 / 1Fichier

\ "fichier"

2 / Alias for an existing remote

\ "alias"

3 / Amazon Drive

\ "amazon cloud drive"

4 / Amazon S3 Compliant Storage Provider (AWS, Alibaba, Ceph, Digital Ocean, Dreamhost, IBM COS, Minio, etc)

\ "s3"

5 / Backblaze B2

\ "b2"

6 / Box

\ "box"

7 / Cache a remote

\ "cache"

8 / Citrix Sharefile

\ "sharefile"

9 / Dropbox

\ "dropbox"

10 / Encrypt/Decrypt a remote

\ "crypt"

11 / FTP Connection

\ "ftp"

12 / Google Cloud Storage (this is not Google Drive)

\ "google cloud storage"

13 / Google Drive

\ "drive"

14 / Google Photos

\ "google photos"

15 / Hubic

\ "hubic"

16 / In memory object storage system.

\ "memory"

17 / Jottacloud

\ "jottacloud"

18 / Koofr

\ "koofr"

19 / Local Disk

\ "local"

20 / Mail.ru Cloud

\ "mailru"

21 / Mega

\ "mega"

22 / Microsoft Azure Blob Storage

\ "azureblob"

23 / Microsoft OneDrive

\ "onedrive"

24 / OpenDrive

\ "opendrive"

25 / OpenStack Swift (Rackspace Cloud Files, Memset Memstore, OVH)

\ "swift"

26 / Pcloud

\ "pcloud"

27 / Put.io

\ "putio"

28 / QingCloud Object Storage

\ "qingstor"

29 / SSH/SFTP Connection

\ "sftp"

30 / Sugarsync

\ "sugarsync"

31 / Tardigrade Decentralized Cloud Storage

\ "tardigrade"

32 / Transparently chunk/split large files

\ "chunker"

33 / Union merges the contents of several upstream fs

\ "union"

34 / Webdav

\ "webdav"

35 / Yandex Disk

\ "yandex"

36 / http Connection

\ "http"

37 / premiumize.me

\ "premiumizeme"

38 / seafile

\ "seafile"

Storage>22

We entered 22 above for Blob storage

Value "sas_url" = "https://specanywheredocstorage.blob.core.windows.net/?sv=2019-10-10&ss=bfqt&srt=sco&sp=rwdlacupx&se=2020-07-04T21:49:19Z&st=2020-07-04T13:49:19Z&spr=https&sig=nNBLB70iJuUdxilMUVGPTfCqCgO6qjGaDSfipoDaRgk%3D"

"

TECHNICAL SPECIFICATION

```
[sample-container]
```

```
type = azureblob
```

```
sas_url = https://specanywheredocstorage.blob.core.windows.net/?sv=2019-10-10&ss=bfqt&srt=sco&sp=rwdlacupx&se=2020-07-04T21:49:19Z&st=2020-07-04T13:49:19Z&spr=https&sig=nNBLB70iJuUdxilMUVGPTfCqCgO6qjGaDSfipoDaRgk%3D
```

```
-----
```

```
F:\rclone\rclone-v1.52.2>rclone mkdir sample-container:container
```

```
F:\rclone\rclone-v1.52.2>rclone ls sample-container:container
```

```
F:\rclone\rclone-v1.52.2>rclone ls sample-container:container
```

```
83783 HVR/TEST1.pdf
```

```
83783 HVR/TEST2.pdf
```

```
83783 HVR/TEST3.pdf
```

```
F:\rclone\rclone-v1.52.2>rclone mount sample-container:container/HVR X:
```

The above command will mount the Azure blob storage onto the new Drive **X:**

As a next step create the following configuration file for FSCrawler:

```
---
name: "filesdb"
fs:
  url: "X:\\"
  update_rate: "2m"
  excludes:
    - "*/~*"
  json_support: false
  filename_as_id: false
  add_filesize: true
  remove_deleted: true
  add_as_inner_object: false
  store_source: false
  index_content: true
  attributes_support: true
  raw_metadata: true
  xml_support: false
  index_folders: true
```

TECHNICAL SPECIFICATION

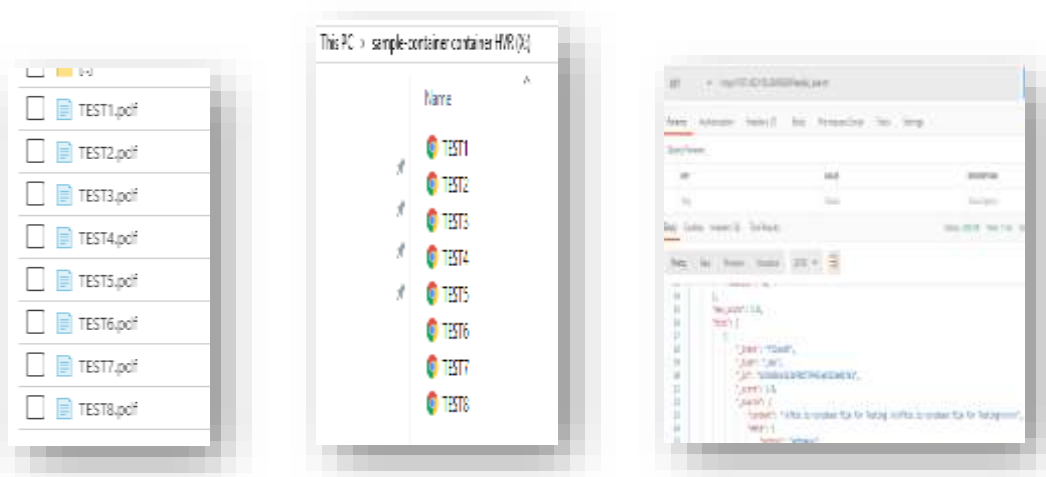
```
lang_detect: false
continue_on_error: false
ocr:
  language: "eng"
  enabled: true
  pdf_strategy: "ocr_and_text"
follow_symlinks: false
elasticsearch:
  nodes:
    - url: "http://137.182.150.28:9303"
  bulk_size: 100
  flush_interval: "5s"
  byte_size: "10mb"
```

The next step is to run a new FSCrawler instance on the new configuration file:

```
F:\FSCrawler\bin\fs-crawler-es7-2.7-SNAPSHOT\bin\fs-crawler filesdb
08:52:49,043 INFO [F.p.e.c.f.c.BootstrapChecks] Memory [Free/Total-Percent]: HEAP [888.8mb/14.3gb-6.1%], RAM [38.5gb/63.0gb-60.26%], Swap [40gb/73.4gb-54.42%].
08:52:50,133 INFO [F.p.e.c.f.c.FsCrawlerCli] attributes.support is set to true but getting group is not available on [windows server 2016].
08:52:50,346 INFO [F.p.e.c.f.FsCrawlerImpl] attributes.support is set to true but getting group is not available on [windows server 2016].
08:52:50,941 INFO [F.p.e.c.f.c.v.ElasticsearchClientV7] Elasticsearch client for version 7.x connected to a node running version 7.5.0
08:52:51,006 INFO [F.p.e.c.f.FsCrawlerImpl] Starting FS crawler
08:52:51,006 INFO [F.p.e.c.f.FsCrawlerImpl] FS crawler started in watch mode. It will run unless you stop it with CTRL+C.
08:52:52,164 INFO [F.p.e.c.f.FsParserAbstract] FS crawler started for [filesdb] for [X:\] every [2m]
08:52:52,674 WARN [o.a.t.p.PDFParser] J2KImageReader not loaded. JPEG2000 files will not be processed.
See https://pdfbox.apache.org/2.0/dependencies.html#jai-image-io
for optional dependencies.
```

Test the data mounted into X: and also verify that the data has been indexed into Elastic index.

Azure ** Local System ** ElasticSearch

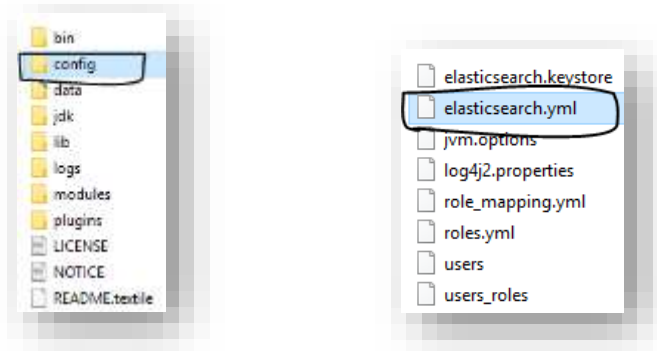


Setting up the pipeline:

Configuring Elasticsearch (ES)

This section does not include the installation of elastic search.

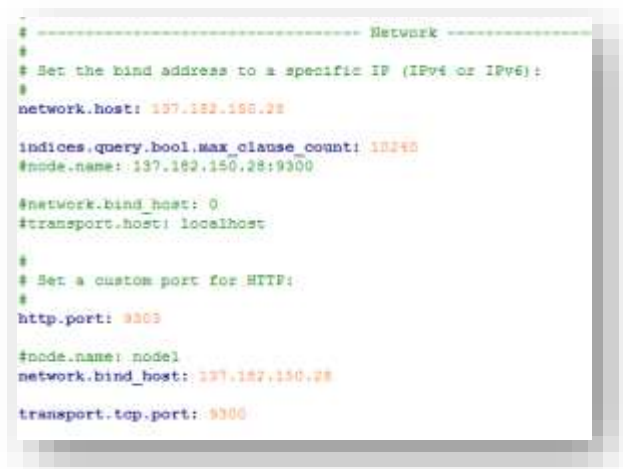
Post installation of ES, the configuration file needs to be adjusted to mention the servers associated to it as shown below:



Mention a cluster name inside the Elasticsearch.yml file



Mention the following network ip and port details of that server:



Here is a sample for configuringm

```
# ----- Discovery -----  
#  
# Pass an initial list of hosts to perform discovery when this node is started:  
# The default list of hosts is ["127.0.0.1", "127.0.0.1"]  
#n  
discovery.seed_hosts: [137.182.150.28]  
*
```

Here is a sample for configuring multiple servers for Elasticsearch:

```
#give your cluster a name.  
cluster.name: my-cluster  
  
#give your nodes a name (change node number from node to node).  
node.name: "es-node-1"  
  
#define node 1 as master-eligible:  
node.master: true  
  
#define nodes 2 and 3 as data nodes:  
node.data: true  
  
#enter the private IP and port of your node:  
network.host: 172.11.61.27  
http.port: 9200  
  
#detail the private IPs of your nodes:  
discovery.zen.ping.unicast.hosts: ["172.11.61.27", "172.31.22.131", "172.31.32.221"]
```

Link to check some basic properties:

<https://logz.io/blog/elasticsearch-cluster-tutorial/>

The logging for elastic comes with a default log4j configuration which can be found here:

log4j2.properties in the */ES../config/..* folder

Setting up the master batch in the Enovia VM:

The following folder structure needs to be maintained in the VM where Enovia is installed for the purpose of this data pipeline:

output

staging

CUP

CustomerUnitPart.tcl

setOPSVariables

The master batch must be able to call all the shell script for each individual type. Above is only 1 example. **The master batch must be executed as a one time activity.**

The setOpsVariables shell must be updated with all the Enovia classpath variables to be able to set the context for execution of the subsequent shell scripts.

This also includes the encrypted pass word of the sys_ops user to be updated.

Example:

```
export HOST_NAME=`uname -n | cut -d "." -f 1`
export MX_HOME=/opt/matrix/3dx/2018x
export MX_BATCHHOME=/opt/matrix/3dx/2018x
export MX_SCRIPTSDIR=$MX_HOME/3dspace/scripts
export MX_RMI=$MX_HOME/3dspace
export MATRIXRMI=$MX_HOME/3dspace
export JAVAPATH=/opt/java1.8
export ENOVIAAPP=$MX_RMI/distrib/enovia
export BINDIR=$ENOVIAAPP/WEB-INF/lib
export MX_JARSDIR=$ENOVIAAPP/WEB-INF/lib
export CLASSES_DIR=$ENOVIAAPP/WEB-INF/classes
export BOOTFILE=MATRIX-R
export CLASSPATH=.:$BINDIR/pgIPMSAPBOM.jar:$BINDIR/jsch-
0.1.32.jar:$BINDIR/EncryptCrypto.jar:$CLASSES_DIR:$CLASSPATH
export CRONDIR=/var/opt/gplm/scripts. $MATRIXRMI/scripts/mxEnv.sh_ctrl-m
```

```
export OPS_TEAM_DIR=/var/opt/gplm/scripts/OPTeam
export OPS_CONTEXTUSER=sys_ops
export OPS_CONTEXTPASS=Wyp0NQBSE5aM6YkNO/umyET/dmQZgiPG02qBZ5Ay0a+6w+nkPOMx5xH0XBu2N8Kr
export OPS_CONTEXTPASS_DECRYPTED=$(JAVAPATH/bin/java -cp $CLASSPATH com.pg.util.EncryptCrypto Decrypt
$OPS_CONTEXTPASS)
```

*The next step is to update the above files with the **Mounted drive** location.*

Example:

This command will mount the container onto the X: on this VM

```
F:\rclone\rclone-v1.52.2>rclone mount csv-container:container X:
```

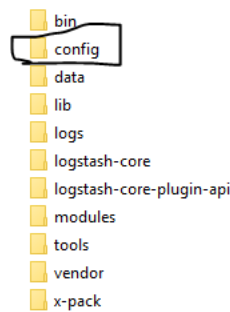
This will enable the batch script to automatically extract data onto this mounted drive.

NOTE:

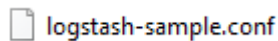
We must mount drives on the source, target VMs. Where, source is the Enovia VM where extraction is scheduled. And the Target VM is the Logstash VM where data will be consumed.

Setting up Logstash:

Post installation of Logstash the following changes need to be made:



Open the config directory, rename and modify the following file”



Update the content of the above configuration file to the following:

```
input {
  file {
    path => "X:/CSV/*.csv"
    start_position => "beginning"
    mode => "read"
    file_completed_action => "delete"
  }
}

filter {
  csv {
    separator => "~"
    columns => [ "type", "name", "revision", "id", "state", "title", "description", "originated", "modified", "ipClass",
"classifiedItem", "secControlClassifiedItem", "originator", "phase", "releasedate", "brand", "filename", "filepath", "company",
"plant", "ownership", "relatedspecFrom", "relatedspecificationTo", "gendoc", "nonrenderabledoc", "otherrefdoc",
"displaytype", "tag", "orgsource", "ShareWithSupplier" ]
  }
}

output {
  elasticsearch {
    hosts => ["http://137.182.150.28:9303"]
    index => "testing"
    document_id => "%{id}"
  }
  stdout {}
}
```

TECHNICAL SPECIFICATION

The highlighted **path** variable must be in-line with the RClone mounted drive locally available. As an example we have mentioned `x:/csv/`

Also the highlighted **hosts** variable must be updated with the elastic cluster. (All IP's of the ES cluster must be mentioned here)

The **index** variable must match the Elasticsearch index created while setting up Elasticsearch DB schema.

The **stdout** variable can point to a log file where the data needs to be captured instead of `stdout`.

Once this is setup Logstash can be started with the command such as:

```
F:/logstash -f <path to logstash configuration file setup above>
```

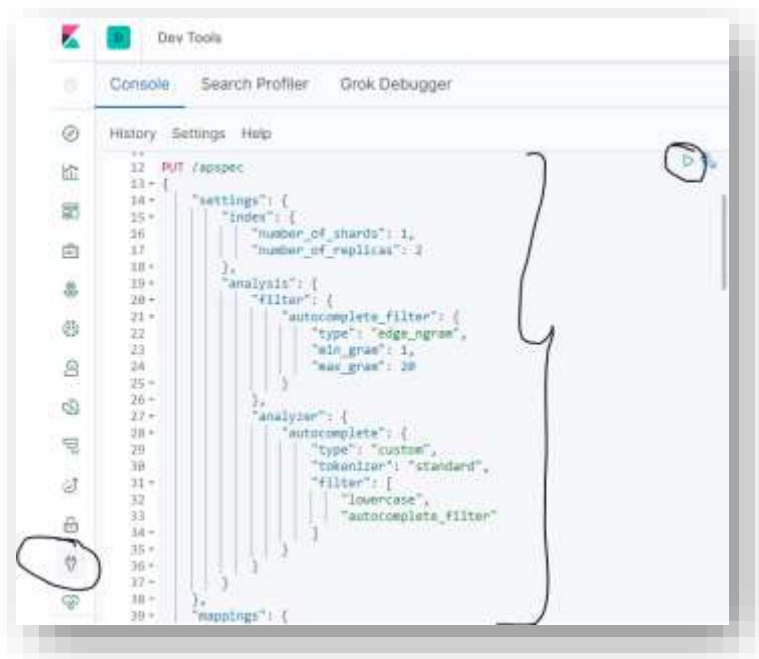
Launch Kibana to setup the elastic schema:

To launch Kibana simply access the root folder and launch Kibana executable from command or your linux prompt.

Launch Kibana on the default port number or provide a port number in the configuration file.

Once Kibana service is running the launch will be successful on your chrome browser as shown below:

You can access "Dev Tools" option to be able to create the elastic search schema. Type in the schema in the space shown below and click on the play button.



Once that is done the right side pane must show a “successful” message to indicate that the Database has been successfully created with the mentioned schema.

The schema has been provided earlier in the document.

As soon as the initial-load has been completed, we need to schedule the Delta master script for every 3 minutes.

Summary of the data pipeline flow:

1. Initial load will generate CSV locally in staging folder and make a copy to Azure blob (mount drive)
2. Logstash will pull the CSV data as and when it identifies a file on the mounted drive. (It takes up to 1 minute for the data to appear in the Mount drive via RClone, this can be adjusted).
3. The data would be continuously synched into Elasticsearch by Logstash (logs of Logstash can be used to verify for any invalid-data failures)
4. Kibana can be used to verify that the data is being consumed by Elasticsearch
5. Once all the initial load data has been pushed to elastic (24 hour period), the delta master batch can be scheduled on linux to fire every 3 minutes.
6. Data can be validated from the logs.

END OF DOCUMENT